JobPilot

Integrated Learning + Build Plan

12 Weeks × 14 Hours/Week — 168 Hours Total

For: **Pranjal Yadav (Red)**

Starting point: Complete AI/ML zero | 3 YoE Backend/Platform @ Oracle

Version 1.0 — 01 April 2026

MVP ships at Week 8. Full system ships at Week 12.

Section 0: The Arithmetic — Why 12 Weeks, Not 8

The PRD estimates 130–160 hours of pure implementation for an engineer already fluent in LangGraph, prompt engineering, RAG, embeddings, and the agent ecosystem. You possess none of these competencies. The minimum viable learning investment before you can write competent agent code is 40–50 hours. That places the realistic total at 170–210 hours.

At 14 hours per week, 8 weeks yields 112 hours — roughly 60–90 hours short. Twelve weeks yields 168 hours, which is achievable only because learning and building are interleaved rather than sequenced. Every concept is learned on the day it is applied. No standalone learning weeks exist.

The Two-Stage Contract

Stage 1 (Weeks 1–8): MVP — Scout, Resume, Application, Tracker, Email Intelligence, Outreach agents. Functional end-to-end pipeline. You can begin applying to jobs.Stage 2 (Weeks 9–12): Full System — Adaptive Resume Engine (4-agent tournament + Mediator), Follow-up Agent, Prometheus + Grafana dashboards, MCP servers, Docker Compose deployment, integration testing.

Section 1: What Transfers From Your Backend Experience

Your Oracle experience provides substantial scaffolding. The following mappings accelerate specific learning modules:

Your Existing Skill	AI/ML Concept	How It Maps
REST API design, Dropwizard	LLM API integration	LLM APIs are HTTP endpoints with JSON payloads. You already know how to call them.
Kubernetes orchestration, OKE	LangGraph state machines	Graph-based state transitions with retry logic. Conceptually similar to K8s controller reconciliation loops.
CI/CD pipelines, TeamCity	Agent pipelines + APScheduler	Sequential and parallel task execution with failure handling. Direct analogue.
Control Plane / Data Plane	Agent architecture	Agents are autonomous services with defined I/O contracts. Your CP/DP mental model applies.
Docker, Helm, Terraform	Docker Compose deployment	Trivial. You already know containerisation.
Prometheus (OCI monitoring)	Prometheus + Grafana	Same stack. Metrics exposition, scrape configs, alerting rules — all familiar.
PostgreSQL (PAIOS)	pgvector, SQLAlchemy	pgvector is an extension on the DB you already run. New concept is vector similarity, not the database.
Security vuln remediation	OAuth 2.0 (Gmail API)	Token management, scope restrictions, credential rotation — security primitives you already practice.

What Has No Backend Equivalent

Prompt engineering (structured system prompts, few-shot examples, output format enforcement), RAG pipelines (embedding generation, vector similarity search, context window management), multi-agent deliberation

(tournament scoring, consensus mechanisms), and LLM-as-judge evaluation patterns. These require deliberate learning — no amount of backend experience shortcuts them.

Section 2: Core Learning Resources

Each resource is cited at the week it is consumed. This section serves as the master reference list.

Domain	Resource	Type	When Used
LLM Fundamentals	Andrej Karpathy — "Intro to LLMs" (YouTube, 1hr)	Video	Week 1, Day 1
LLM Fundamentals	OpenAI Tokenizer (tiktoken playground)	Tool	Week 1, Day 2
Prompt Engineering	Anthropic Prompt Engineering docs (docs.anthropic.com)	Docs	Week 1, Day 3–4
Prompt Engineering	OpenAI Prompt Engineering Guide	Docs	Week 1, Day 5
Python for AI	LangChain official docs — quickstart	Docs	Week 2, Day 1
LLM APIs	DeepSeek API docs, Qwen (DashScope) API docs	Docs	Week 2, Day 2–3
Agent Frameworks	LangGraph official tutorial (langchain-ai.github.io)	Docs	Week 4, Day 1–3
Agent Frameworks	Harrison Chase — LangGraph walkthrough (YouTube)	Video	Week 4, Day 1
RAG & Vectors	pgvector docs + LangChain RAG tutorial	Docs	Week 3, Day 1–4
RAG & Vectors	Pinecone Learning Center — "What are Embeddings?"	Article	Week 3, Day 1
Gmail API	Google Gmail API Python quickstart	Docs	Week 5, Day 1
MCP Protocol	Anthropic MCP specification (modelcontextprotocol.io)	Docs	Week 10, Day 4
Evaluation	LangSmith / LangFuse docs (observability)	Docs	Week 11, Day 6
Playwright	Playwright Python docs — async API	Docs	Week 5, Day 3
LaTeX	pdflatex CLI reference, Overleaf guides	Docs	Week 3, Day 5–7

Section 3: Week-by-Week Plan

Every row specifies what you learn and what you build on that day. The two columns are interleaved: read/watch the concept, then immediately write the code that applies it. Two hours per day, seven days a week.

STAGE 1: MVP (Weeks 1–8) — Functional Job Application Pipeline

Scout → Resume → Application → Tracker → Email Intelligence → Outreach. End-to-end pipeline with Telegram HITL. You can begin applying to real jobs at the end of Week 8.

Week 1: LLM Fundamentals + Project Scaffolding

Phase: *Foundation* | **Hours:** 14 hrs

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	LLM Concepts	Watch Karpathy "Intro to LLMs." Understand tokens, context windows, temperature, top-p.	Set up project repo. docker-compose.yml with PostgreSQL. FastAPI /health endpoint.	Repo initialised
2	Tokenisation	tiktoken playground: paste your resume, observe token count. Token limits and cost implications.	Install Python deps (langchain, openai, httpx). Create config.yaml parser and .env.	Config system
3	Prompt Eng I	Anthropic docs: system prompts, role prompts, output format enforcement, XML tag patterns.	Write JD scoring system prompt (PRD §5.2). Test against 3 real JDs via DeepSeek API.	Scoring prompt v1
4	Prompt Eng II	Few-shot prompting, chain-of-thought. Structured JSON output enforcement.	Write JD keyword extraction prompt (PRD §6.2). Test extraction against 3 real JDs.	Extraction prompt
5	Cascading LLM	DeepSeek and Qwen API docs. Rate limits, error codes, retry semantics.	Implement CascadingLLM class (PRD §4.2). 5-tier chain with Ollama terminal.	Cascade client
6	Telegram Bot	python-telegram-bot v20: handlers, inline keyboards, callback queries.	Create Telegram bot. Implement send_job_notification() with Apply/Skip/Details buttons.	Telegram bot live
7	Integration	Review and refactor all Week 1 code.	Wire CascadingLLM + scoring prompt + Telegram. Manual test: paste JD → scored notification.	End-to-end test

Week 2: Scout Agent — Job Discovery

Phase: *Scout Agent Build* | **Hours:** 14 hrs

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable

1	Web Scraping	httpx async basics. BeautifulSoup DOM parsing. Rate limiting patterns.	Lever generic scraper: {company}.lever.co/api/postings JSON. Parse title, location, description.	Lever scraper
2	ATS Scrapers	Greenhouse API patterns. Ashby page structure. feedparser for RSS/Atom.	Greenhouse scraper. RSS auto-detection probe. feedparser integration.	3 ATS scrapers
3	Title Matching	Fuzzy string matching: Levenshtein distance, token overlap. rapidfuzz library.	Title pre-filter (PRD §5.2 Stage 1). Hardcoded title list + fuzzy matching.	Title filter
4	Deduplication	SHA-256 hashing for dedup. PostgreSQL index for O(1) lookup.	Dedup pipeline (PRD §5.2 Stage 4). PostgreSQL seen-postings table. Geographic filter.	Dedup + geo filter
5	Full Pipeline	Pipeline assembly: scrape → title filter → LLM score → dedup → threshold → notify.	Wire full Scout Agent. APScheduler: 10min tier-1, 4hr tier-2.	Scout Agent v1
6	LinkedIn/Naukri	LinkedIn Jobs scraping constraints. Naukri API/scraping patterns.	LinkedIn jobs scraper (httpx). Naukri scraper. Integrate into Scout.	All sources live
7	Testing	Edge cases: empty JDs, non-English postings, malformed HTML.	test_scout.py. Run against 10 real career pages. Fix failures.	Scout tested

Week 3: RAG + Resume Agent

Phase: *RAG & Vectors + Resume Build* | **Hours:** 14 hrs

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	Embeddings	What are embeddings. Cosine similarity. Pinecone Learning Center article.	Install pgvector on PostgreSQL. Create embeddings table. Test with sentence-transformers.	pgvector running
2	RAG Pipeline	LangChain RAG tutorial: loading, splitting, embedding, retrieval.	Embed your resume + 5 real JDs. Query by cosine similarity.	RAG prototype
3	RAG in Scout	Hybrid scoring: LLM + embedding similarity for relevance.	Add pgvector pre-scoring to Scout Agent. Compare against LLM-only.	Hybrid scoring
4	LaTeX Parsing	Your .tex template: \section, \resumeSubheading, \resumeItem. Regex extraction.	LaTeX parser: sections, bullets, skills into structured intermediate representation.	LaTeX parser
5	Bullet Rewriting	Resume rewriting prompt constraints (PRD §6.2 Step 3). No-fabrication rules.	Bullet rewriting via CascadingLLM. Harvard action verb injection. Test on 3 JD pairings.	Rewriter v1
6	LaTeX Compile	pdflatex CLI. Error parsing. Auto-fix for unescaped &, %, $.	texlive in Docker. compile_latex() function (PRD §6.2 Step 6). Test compilation.	Compiler working

7	ATS Scoring	Keyword match rate. Bullet reordering. Skills section reordering.	ATS score estimation. Reordering logic. Resume Agent end-to-end test.	Resume Agent v1

Week 4: LangGraph + Tracker + Application Agent

Phase: *Agent Orchestration* | **Hours:** 14 hrs

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	LangGraph I	LangGraph tutorial: StateGraph, nodes, edges, conditional routing. Harrison Chase video.	Minimal LangGraph: 2 nodes (Scout → Resume) with shared state.	First graph runs
2	LangGraph II	HITL checkpoints, interrupt_before, human approval nodes. State persistence.	Add Telegram HITL checkpoint between Scout and Resume nodes.	HITL checkpoint
3	LangGraph III	Parallel execution, fan-out/fan-in. Error handling and retry.	Expand graph: Scout → [HITL] → Resume → [HITL] → Application (stub). Full state schema.	3-node graph
4	Tracker Agent	openpyxl: workbooks, sheets, conditional formatting, column sizing.	Tracker Agent (PRD §9). 30-column schema. Auto-populate from Scout + Resume state.	Tracker v1
5	Tracker Polish	Conditional formatting rules: red/yellow/green rows.	Conditional formatting (PRD §9.3). Daily summary Telegram message at 21:00.	Tracker polished
6	Application	Lever POST /postings/{id}/apply. Gmail gmail.send for email applications.	Application Agent (PRD §7). Lever API + email + manual fallback.	Application Agent
7	Integration	End-to-end: Scout → Resume → Application → Tracker.	Run pipeline against 5 real postings. Fix integration bugs.	4-agent pipeline

Week 5: Gmail OAuth + Outreach Agent

Phase: *Application Completion* | **Hours:** 14 hrs

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	Gmail OAuth	Google Cloud Console: project, Gmail API, OAuth credentials. OAuth 2.0 flow.	setup_gmail_oauth.py. Obtain refresh token. Test gmail.readonly scope.	Gmail auth done
2	Gmail Send	Gmail API: messages.send(), MIME construction, attachments.	Email application submission via Gmail API. Resume PDF attachment.	Email submission
3	Playwright	Playwright Python async: browser launch, selectors, form filling.	Playwright install. LinkedIn Easy Apply prototype. Rate limits.	Playwright basics

4	Form Filling	applicant_profile.yaml auto-population. ATS form field mapping.	Form auto-population (PRD §7.3). Test against Lever and Greenhouse forms.	Auto-fill working
5	Confirmation	Confirmation gate flow (PRD §7.2). Telegram approval before submission.	Telegram Confirm/Cancel/View Resume buttons. Wire into LangGraph.	Confirmation gate
6	Outreach Prep	Apollo.io API: search_people, free tier (50/month). Hunter.io API.	Apollo.io account. find_recruiters_apollo() (PRD §8.1). Test with 3 companies.	Apollo integration
7	Outreach Agent	Dual-variant messages (PRD §8.2). Rate limits (PRD §8.3).	Outreach Agent: 2 variants per recruiter. Telegram selection UI.	Outreach Agent v1

Week 6: Email Intelligence Agent

Phase: *Email Classification* | **Hours:** 14 hrs

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	Gmail Sync	users.history().list(), historyId incremental sync. Label management.	Incremental email sync (PRD §10.1). Store historyId in PostgreSQL.	Email sync
2	Pre-filters	Domain matching vs tracker. Regex keyword classification (PRD §10.2).	Stage 1 (domain filter) and Stage 2 (keyword pre-filter). Test against real inbox.	Pre-filters done
3	LLM Classify	8-category taxonomy (PRD §10.2). Confidence thresholds.	Classification prompt. Stage 3 LLM classification via CascadingLLM.	Classifier done
4	Tracker Update	Auto-updating tracker from classifications. Status transitions.	Email Intelligence → Tracker Agent. Auto-label Gmail (PRD §10.4).	Auto-updates
5	Feedback Store	Outcome weight system (PRD §10.3). Adaptive Resume Engine data schema.	Feedback data collection. Per-application outcome weights in PostgreSQL.	Feedback store
6	Privacy	Privacy constraints (PRD §10.5). Classification-only persistence.	Privacy audit. No raw email storage. Token encryption.	Privacy audit
7	Full Pipeline	All 6 agents: Scout → Resume → Application → Tracker → Email Intel → Outreach.	End-to-end test with 10 real postings. Fix integration issues.	6-agent pipeline

Week 7: Pipeline Hardening

Phase: *Stabilisation* | **Hours:** 14 hrs

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	Error Handling	Retry strategies. Graceful degradation. Dead letter queues.	Structured error handling for all 6 agents. Exponential backoff.	Error handling

2	Rate Limits	Per-source limits: LinkedIn 30/day, Apollo 50/mo, Gmail 250/day.	Rate limiter middleware. Token bucket per service. Limit alerts.	Rate limiters
3	Scheduling	APScheduler cron. Job persistence. Timezone (Asia/Kolkata).	All APScheduler jobs: Scout 10min/4hr, Email Intel 5min, summary 21:00.	Scheduler done
4	State Recovery	LangGraph checkpointing. PostgreSQL-backed persistence.	Checkpoint persistence. Test crash recovery.	State persistence
5	Telegram UX	Review all Telegram interactions for clarity and formatting.	Polish messages. Add /status, /pause, /resume commands.	Telegram polished
6	Logging	Structured logging. Log levels. Correlation IDs.	Logging across all agents. Log every LLM call with tier, latency, tokens.	Logging done
7	Bug Fixes	Review all open issues from Weeks 1–6.	Fix bugs. Run pipeline 24hr unattended. Monitor logs.	Stable pipeline

Week 8: MVP Ship + Docker Compose

Phase: *MVP Deployment* | **Hours:** 14 hrs

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	Dockerfile	Multi-stage builds. Python + TeX Live. .dockerignore.	Dockerfile with pdflatex, Python deps, Playwright browsers.	Dockerfile
2	Compose	docker-compose: services, volumes, networking, healthchecks.	docker-compose.yml: app + PostgreSQL. Shared resume volume.	Compose file
3	Data Seeding	Schema management. Initial data (companies.yaml, Harvard verbs).	init_database.sql. Seed 200 companies with ATS annotations.	DB seeded
4	End-to-End	docker-compose up → full system. Verify all agents.	Full integration: discover → score → notify → approve → tailor → apply → track.	Pipeline verified
5	Documentation	README: quickstart, config, architecture, troubleshooting.	README.md. Document all env vars. Architecture diagram.	README done
6	Live Trial	Run against real job boards for a full day.	Apply to 5–10 jobs with Telegram approval. Verify tracker accuracy.	LIVE DEPLOYMENT
7	Retrospective	Review: what worked, what broke, what needs fixing.	Fix critical issues. Adjust scoring threshold if FP rate > 15%.	MVP SHIPPED

✅ MVP Milestone (End of Week 8)

Pipeline is live. Scout discovers jobs, Resume Agent tailors LaTeX, Application Agent submits with approval, Tracker auto-populates, Email Intelligence classifies, Outreach contacts recruiters. You are actively applying. 112 hours consumed.

STAGE 2: Full System (Weeks 9–12) — Intelligence + Observability

Adaptive Resume Engine, Follow-up Agent, Prometheus + Grafana, MCP servers. The system becomes self-improving and fully observable.

Week 9: Adaptive Resume Engine

Phase: *Multi-Agent Tournament* | **Hours:** 14 hrs

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	Feedback Data	Positive/negative signal queries (PRD §12.2). Outcome weights.	SQL feedback collection queries. Feedback aggregation module.	Feedback queries
2	Tournament I	4-agent architecture (PRD §12.3). Parallel agents. Role specialisation.	Keyword Agent and Structure Agent. Base resume + feedback + system prompt.	2 tournament agents
3	Tournament II	Narrative Agent (phrasing). Market Agent (emerging JD trends).	Narrative and Market Agents. Wire all 4 for parallel execution.	4 tournament agents
4	Judge + Score	LLM-as-judge. 5-dimension rubric (PRD §12.4). Truthfulness 2x weight.	Judge Agent. Score proposals. Determine winner and margin.	Judge scoring
5	Deliberation	Mediator Agent: triggered when margin < 5. Merge synthesis (PRD §12.4).	Mediator Agent. Re-score merged version. 3-candidate selection.	Deliberation done
6	Versioning	Git-like history (PRD §12.5). Metadata. Auto-changelog.	version_manager.py. resume_versions/ directory. Symlinks. changelog.md.	Version control
7	Safety	Guardrails (PRD §12.6): no-fabrication, performance floor, /revert, /pin.	All guardrails. APScheduler 06:00/18:00. Cold start logic.	ARE complete

Week 10: Follow-up Agent + MCP Servers

Phase: *Follow-up + MCP* | **Hours:** 14 hrs

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	Follow-up	Trigger logic (PRD §11.1). Day 7 + Day 14 SQL. Skip rules.	Follow-up candidate query. APScheduler daily 10:00 AM IST.	Follow-up trigger
2	Email Draft	Follow-up prompt (PRD §11.2). Prohibited phrases. Plain text.	LLM follow-up drafting. Day 7 (4 sentences) and Day 14 (2, final).	Draft generation
3	Approval Flow	Telegram UI (PRD §11.3). Send/Edit/Skip/Delay buttons.	Telegram follow-up approval. Delay 3 days. Gmail API send.	Follow-up Agent
4	MCP Basics	MCP spec: servers, tools, resources. Streamable HTTP. FastMCP.	jobpilot-scout MCP server (PRD §14.1). search_jobs(), get_job_details().	Scout MCP

5	MCP Resume	MCP tool schemas. Input/output contracts.	jobpilot-resume MCP server (PRD §14.2). tailor_resume(), estimate_ats_score().	Resume MCP
6	MCP Outreach	MCP testing. Client integration.	jobpilot-outreach MCP server (PRD §14.3). find_recruiters(), draft_outreach().	3 MCP servers
7	Full Graph	9-agent LangGraph verification.	Update LangGraph with Follow-up + ARE nodes. Full integration test.	9-agent graph

Week 11: Prometheus + Grafana + Observability

Phase: *Monitoring* | **Hours:** 14 hrs

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	Metrics Def	prometheus_client: Counter, Gauge, Histogram. /metrics endpoint.	Define 15+ metrics (PRD §13.2). Expose /metrics via FastAPI.	Metrics endpoint
2	Prometheus	prometheus.yml scrape config. alert_rules.yml.	Prometheus in docker-compose. 15s scrape. 4 alert rules.	Prometheus live
3	Grafana I	Provisioning: datasources, dashboard JSON. Panel types.	Grafana in docker-compose. Prometheus datasource. Funnel dashboard.	Funnel dashboard
4	Grafana II	PromQL: conversion rates, timing distributions, cost tracking.	4 remaining dashboards: daily, resume evolution, LLM cost, outreach.	5 dashboards
5	Alerting	Grafana alerting: channels, conditions, silence rules.	Telegram as alert channel. Test all 4 rules.	Alerts working
6	LangSmith	LangSmith/LangFuse: LLM tracing, token usage, latency.	LangSmith tracing in CascadingLLM. Tag by agent and task.	Tracing live
7	Eval Suite	LLM-as-judge for quality validation (quality.py).	quality.py: resume rewriting quality scoring. Threshold retry.	Eval pipeline

Week 12: Deployment + Testing + Ship

Phase: *Final Deployment* | **Hours:** 14 hrs

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	Docker Polish	Health checks, restart policies, volumes, log rotation.	Final docker-compose: app + PostgreSQL + Prometheus + Grafana + Ollama.	Final compose
2	Testing I	Integration test design: agent pairs, pipeline, errors.	test_resume.py, test_cascade_llm.py, test_email_intelligence.py.	Test suite I
3	Testing II	ARE testing: tournament, deliberation, revert.	test_adaptive_resume.py, test_outreach.py,	Test suite II

			test_follow_up.py, test_metrics.py.	
4	Load Test	50 concurrent postings. Cascading LLM under rate limit.	Load test: 50 synthetic postings. Verify no data loss, correct dedup.	Load tested
5	Docs + VPS	Hetzner CX22 setup (optional). SSH, Docker, firewall.	Update README. Troubleshooting guide. Optional VPS deploy.	Docs complete
6	Soak Test	24-hour unattended run. Prometheus dashboards.	24hr soak test. Review metrics. Fix anomalies. Verify ARE fires.	Soak test passed
7	Ship	Final PRD requirements review. Gap analysis.	Tag v1.0 release. Update changelog.	JOBPILOT v1.0

✅ Full System (End of Week 12)

All 9 agents. Adaptive Resume Engine with 12hr cycles. Prometheus + Grafana (5 dashboards). 3 MCP servers. Follow-up Agent (day 7 + 14). Docker Compose deployment. 168 hours consumed.

Section 4: SDE Interview Competence Map

Each component you build maps directly to SDE interview topics.

Interview Domain	What You Built	Interview Application
System Design	Multi-agent orchestration via LangGraph with state persistence, HITL checkpoints, cascading fallback	"Design a job matching system" — you built one. Agent decomposition, state machines, HITL, scaling.
API Design	CascadingLLM with retry, Lever/Greenhouse/Apollo integrations, MCP server tools	"Design an API gateway with fallback" — circuit breakers, timeouts, idempotency.
Data Modelling	30-column tracker, pgvector embeddings, feedback store, version metadata	"Design a schema for X" — normalisation, indexes, JSON vs relational trade-offs.
Distributed Systems	Concurrent agents, APScheduler coordination, Docker multi-service	"Handle concurrent writes?" — transaction isolation, locking, idempotent operations.
ML/AI Integration	RAG pipeline, LLM-as-judge, multi-agent tournament consensus	"Integrate LLMs into a product?" — prompt eng, embeddings, cost optimisation, eval loops.
Monitoring	15+ Prometheus metrics, 5 Grafana dashboards, 4 alert rules, LangSmith	"Monitor production services?" — concrete metrics, thresholds, dashboards you built.
Security	OAuth 2.0 (Gmail), no-raw-email-storage, token encryption	"Handle credentials?" — OAuth flow, refresh tokens, encrypted storage, least privilege.
Testing	Unit per agent, integration pairs, load test (50 concurrent), 24hr soak	"Test a complex system?" — test pyramid, soak testing, LLM output validation.

Section 5: Cumulative Hours Breakdown

Week	Focus	Learn (hrs)	Build (hrs)	Total
1	LLM Fundamentals + Scaffolding	7	7	14
2	Scout Agent	4	10	14
3	RAG + Resume Agent	6	8	14
4	LangGraph + Tracker + Application	5	9	14
5	Gmail + Outreach Agent	3	11	14
6	Email Intelligence Agent	3	11	14
7	Pipeline Hardening	2	12	14
8	MVP Ship + Docker Compose	1	13	14
	STAGE 1 SUBTOTAL	**31**	**81**	**112**
9	Adaptive Resume Engine	4	10	14
10	Follow-up Agent + MCP Servers	4	10	14
11	Prometheus + Grafana	3	11	14
12	Deployment + Testing + Ship	1	13	14
	STAGE 2 SUBTOTAL	**12**	**44**	**56**
	GRAND TOTAL	**43**	**125**	**168**

Learning constitutes 25.6% of total time (43 of 168 hours). Front-loaded: Weeks 1–3 spend ~40–50% learning, Weeks 7–12 spend under 15%. By Week 8, you are building almost exclusively from acquired competence.

Section 6: Plan-Specific Risks

Risk	Prob.	Mitigation
LangGraph steeper than estimated	**High**	Week 4 has 3 dedicated days. If struggling by Day 3: simplify to plain Python orchestrator (while loop + dispatch) for MVP. Migrate to LangGraph in Stage 2.
LinkedIn scraping triggers restriction	**Medium**	Apollo.io is primary. LinkedIn is fallback, 30/day. Use throwaway account for dev. Never automate InMail.
2 hrs/day insufficient for complex days	**High**	Bank time from lighter days. Or do a 3-hour Saturday session for LangGraph-heavy days.
Free-tier LLM quality too low for rewriting	**Medium**	CascadingLLM routes to Claude Sonnet for quality-critical tasks. Zero-budget: accept ~15–20% lower phrasing sophistication.
Email misclassifies interview as rejection	**Low**	0.85 confidence threshold routes ambiguous to Telegram. No auto-action below threshold.
ARE degrades resume quality	**Medium**	Performance floor: <10% acceptance for 3 cycles → auto-revert. /pin freezes known-good version.
Motivation drops at week 10	**Medium**	MVP ships at Week 8 and is immediately useful. Weeks 9–12 enhance a system you use daily.

Section 7: Recommended Daily Routine

A consistent 2-hour block outperforms scattered fragments. Structure each session:

Block	Duration	Activity
Orientation	5 minutes	Open this document. Read today's row. Identify Learn and Build tasks.
Learn Block	25–40 min	Read docs, watch the video, or study the code pattern in the Learn column.
Build Block	70–85 min	Write the code in the Build column. Test it. Commit to git.
Close	5 minutes	git commit with week/day reference (e.g., "W3D4: LaTeX parser"). Note blockers.

The 25–40 / 70–85 split is not rigid. Familiar concepts (Docker in Week 8): 10 min learn, 100 min build. Novel concepts (LangGraph in Week 4): 40 min learn, 70 min build. Total is always 120 minutes.